RELATÓRIO DE AUDITORIA DE SEGURANÇA

Auditoria de Segurança & Código: Demo SaaS Platform

Data da Auditoria: 2026-11-09 • Escopo: Backend API, Frontend Next.js e Infraestrutura Docker

Stack Tecnológica Detectada:

Linguagem: TypeScript / Node.js | **Framework:** Next.js (App Router) + Express API | **ORM/DB:** Prisma ORM + PostgreSQL
Auth: NextAuth.js (JWT session) | **Frontend:** React 19 + Tailwind CSS | **Deploy/Infra:** Docker Compose + GitHub Actions

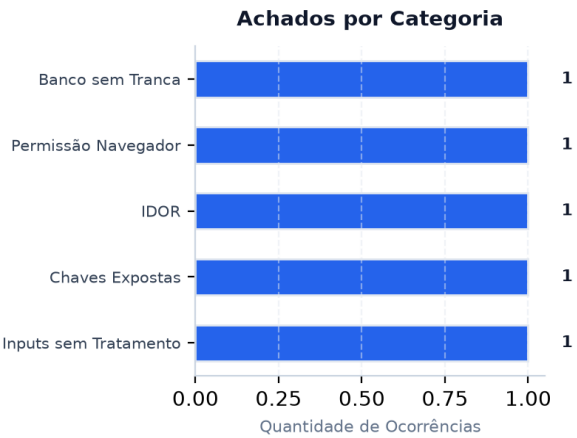
Nota Metodológica de Auditoria:

Varredura focada em isolamento multitenant, RLS/filtros de query, autorização simétrica frontend/backend, prevenção de IDOR, sanitização de segredos e higienização de inputs no DOM e e-mails.

Resumo Executivo

Total	Crítica	Alta	Média	Baixa	Informativa
7	1	2	2	1	1





Pontos Fortes (Mecanismos Corretos & Protegidos)

Evidências de áreas examinadas que apresentaram defesas robustas e isolamento eficaz:

• **[PROTEGIDO] Rotas de Faturamento (Billing):** Todas as consultas de invoices validam estritamente o tenantId extraído do token JWT autenticado. *(Evidência: src/controllers/billing.controller.ts:45-80)*

• **[PROTEGIDO] Hashing de Senhas:** Utilização correta de bcrypt com salt rounds = 12 em todas as operações de cadastro e redefinição. *(Evidência: src/services/auth.service.ts:18)*

Riscos Centrais (Pontos Fracos)

• **[RISCO]** Falta de validação de tenantId no endpoint de exportação de relatórios CSV.

• **[RISCO]** Rota de remoção de usuários sem verificação de role administrativa no backend.

• **[RISCO]** Default de segredo JWT inseguro no docker-compose.yml.

Achados Detalhados por Categoria

Varredura linha por linha dos achados verificados no código real:

Sev.	Arquivo : Linha	Categoria & Descrição do Problema
CRÍTICA	`src/routes/reports.ts:78`	**Exportação CSV sem filtro de inquilino (Tenant Leak)** Categoria: Banco sem Tranca A query de exportação busca registros sem filtrar pelo tenantId da sessão do usuário autenticado. **Explorabilidade:** *Qualquer usuário logado pode chamar GET /api/reports/export-all e baixar dados de todos os inquilinos cadastrados.* **Correção Sugerida:** Adicionar where: { tenantId: req.user.tenantId } obrigatoriamente.
ALTA	`src/routes/users.ts:142`	**Exclusão de usuário sem verificação de role no backend** Categoria: Permissão no Navegador O endpoint de exclusão checa apenas autenticação mas não exige autorização RBAC de administrador. **Explorabilidade:** *O frontend oculta o botão com {user.role === 'admin'}, mas requisição manual via cURL com token comum executa a exclusão com sucesso.* **Correção Sugerida:** Inserir middleware requireRole('admin') na definição da rota.
ALTA	`src/routes/profile.ts:55`	**Modificação de perfil de outro usuário por ID** Categoria: IDOR O handler de atualização confia no ID passado na rota sem comparar com o ID autenticado na sessão. **Explorabilidade:** *Usuário A altera o ID na URL para o ID de usuário B e sobrescreve telefone, e-mail e preferências alheias.* **Correção Sugerida:** Utilizar req.user.id diretamente na cláusula where ou validar if (req.params.id !== req.user.id) return 403.

MÉDIA	`docker-compose.yml:24`	**Fallback de segredo JWT fraco e previsível** Categoria: Chaves Expostas Variável de ambiente com fallback padrão fraco e ausência de verificação no startup. **Explorabilidade:** *Se o container subir sem .env em produção, qualquer atacante pode forjar tokens JWT válidos.* **Correção Sugerida:** Remover o fallback e adicionar verificação em src/index.ts que encerra a aplicação se JWT_SECRET não estiver definido.
MÉDIA	`src/components/UserProfile.tsx:64`	**Renderização de bio de usuário com dangerouslySetInnerHTML** Categoria: Inputs sem Tratamento Conteúdo vindo de entrada de usuário renderizado no DOM sem sanitização prévia. **Explorabilidade:** *Injeção de scripts maliciosos através do campo de biografia, executando roubo de sessão de outros visitantes.* **Correção Sugerida:** Sanitizar o HTML usando DOMPurify.sanitize(user.bio) ou utilizar renderização como texto comum.

Recomendações Priorizadas

Plano de mitigação baseado na severidade e complexidade:

[P1] Correção Imediata do Isolamento de Inquilino: Aplicar filtro compulsório de tenantId na rota de exportação CSV e auditar todos os findMany do Prisma.

[P2] Alinhamento de Permissões Backend e Proteção IDOR: Implantar requireRole('admin') nas rotas sensíveis e validar posse nos updates por ID.

[P3] Eliminação de Segredos Padrão e Sanitização de DOM: Remover fallback de segredos no docker-compose e aplicar DOMPurify na exibição de bios.